



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Московский государственный технический
университет имени Н.Э. Баумана национальный исследовательский
университет» (МГТУ им. Н.Э. Баумана)

Лабораторная работа №3

Исследование методов циклического кодирования

Студенты группы ИУ1-41Б

Соин А. Д.
Кочнева К. Д.
«14» мая 2026 г.

Преподаватель

Кузнецов М. А.
«_____» _____ 2026 г.

Москва, 2026

Содержание

1	Реализация побитного циклического кодирования	2
2	Реализация побитного “исключающего или”	3
3	Контрольные вопросы	3
	Приложение 1. Программный код	5

1 Реализация побитного циклического кодирования

В ходе выполнения лабораторной работы был написан необходимый программный код на языке matlab.

В результате выполнения кода был получен следующий вывод:

```
Кодовый полином:
1 0 0 1 1
=====
Исходное сообщение (dec): 46
Исходное сообщение (bin):
0 0 1 0 1 1 1 0
=====
CRC остаток:
1 0 1 1
=====
Закодированное сообщение:
0 0 1 0 1 1 1 0 1 0 1 1
=====
Модель ошибки:
0
0
0
0
0
0
0
0
1
0
0
0
0
=====
Полученное сообщение:
0 0 1 0 1 1 1 1 1 0 1 1
=====
Декодированное сообщение:
0 0 1 0 1 1 1 1
=====
Ошибка обнаружена: 1
=====
Синдром ошибки: 1
=====
Таблица синдромов
  BitNumber      Syndrome
  -----
      1      0  1  0  1
      2      1  1  0  0
      3      0  0  0  1
      4      1  1  1  0
```

5	0	0	0	0
6	0	1	1	1
7	1	1	0	1
8	1	0	0	0
9	1	0	1	1
10	1	0	1	1
11	1	0	1	1
12	1	0	1	1

Ошибочный бит: 5

Ошибка моделируется и обнаруживается правильно.

2 Реализация побитного “исключающего или”

Исключающее или было реализовано на основе цикла for.

XOR-кодирование

0 0 1 0 1 1 1 0

=====

Ошибка в одном бите

Полученное сообщение:

0 0 1 1 1 1 1 0

Ошибка обнаружена

=====

Ошибка в двух битах

Полученное сообщение:

0 0 1 1 1 0 1 0

Ошибка НЕ обнаружена

В результате выполнения выяснено, что XOR-контроль четности обнаруживает ошибки только при нечетном количестве искаженных бит. При четном количестве ошибок обнаружение невозможно.

3 Контрольные вопросы

1. В чем суть циклического кодирования?

Циклическое кодирование — это метод обнаружения и/или исправления ошибок при передаче данных, основанный на представлении двоичных последовательностей в виде полиномов над полем $GF(2)$.

Основная идея метода:

- сообщение интерпретируется как полином;
- сообщение делится на порождающий полином;
- остаток от деления (CRC) добавляется к сообщению;
- на приемной стороне выполняется повторное деление;
- если остаток ненулевой — обнаружена ошибка.

Циклические коды эффективно обнаруживают одиночные, пакетные и большинство многобитовых ошибок.

2. Для каких задач могут применяться методы циклического кодирования?

Методы циклического кодирования применяются в системах передачи и хранения данных для контроля целостности информации.

Основные области применения:

- компьютерные сети;
- телекоммуникационные системы;
- мобильная и спутниковая связь;
- интерфейсы передачи данных (USB, CAN, SATA);
- устройства хранения данных;
- архиваторы и файловые форматы;
- микроконтроллеры и встроенные системы.

CRC широко используется благодаря высокой надежности обнаружения ошибок и простоте реализации.

3. В чем отличие побитного от побайтного метода циклического кодирования, в каких случаях какому методу необходимо отдавать приоритет?

Побитный метод выполняет обработку данных по одному биту:

- требует меньше памяти;
- проще реализуется аппаратно;
- подходит для потоковой обработки;
- работает медленнее.

Побайтный метод выполняет обработку сразу байтами:

- обладает высокой скоростью работы;
- эффективен при обработке больших объемов данных;
- требует дополнительной памяти для таблиц.

Побитный метод предпочтителен для аппаратной реализации и систем с ограниченными ресурсами, а побайтный — для высокоскоростной программной обработки данных.

4. Что такое порождающий полином?

Порождающий полином — это двоичный полином, определяющий структуру циклического кода и используемый для вычисления контрольной суммы CRC.

Например, для CRC-4-ITU используется полином:

$$g(x) = x^4 + x + 1$$

В двоичном виде:

$$[1\ 0\ 0\ 1\ 1]$$

Степень порождающего полинома определяет количество контрольных бит, добавляемых к сообщению.

Приложение 1. Программный код

```
1      clc;
2      clear;
3      close all;
4
5      poly = [1 0 0 1 1];
6
7      crcGen = comm.CRCGenerator( ...
8          'Polynomial', poly, ...
9          'ChecksumsPerFrame', 1);
10     crcDet = comm.CRCDetector( ...
11         'Polynomial', poly, ...
12         'ChecksumsPerFrame', 1);
13
14     msg_dec = 46;
15
16     msg = de2bi(msg_dec, 8, 'left-msb');
17     encoded = step(crcGen, logical(msg));
18
19     msg_len = length(msg);
20     enc_len = length(encoded);
21     redundancy = enc_len - msg_len;
22
23     crc_bits = encoded(end-3:end);
24
25     err_pos = length(encoded) - 5 + 1;
26     error_dec = 2^(5-1);
27     error_vec = de2bi(error_dec, length(encoded), 'left-msb');
28     received = bitxor(encoded, logical(error_vec));
29     [decoded, errFlag] = step(crcDet, received);
30
31     disp('=====');
32     disp('Кодовый полином:');
33     disp(num2str(poly));
34     disp('=====');
35     disp(['Исходное сообщение (dec): ', num2str(msg_dec)]);
36     disp('Исходное сообщение (bin):');
37     disp(num2str(msg));
38     disp('=====');
39     disp('CRC остаток:');
40     disp(num2str(crc_bits));
41     disp('=====');
42     disp('Закодированное сообщение:');
43     disp(num2str(encoded));
44     disp('=====');
45     disp('Модель ошибки:');
46     disp(num2str(error_vec));
47     disp('=====');
48     disp('Полученное сообщение:');
```

```

49     disp(num2str(received));
50     disp('=====');
51     disp('Декодированное сообщение:');
52     disp(num2str(decoded));
53     disp('=====');
54     disp(['Ошибка обнаружена: ', num2str(errFlag)]);
55
56     [~, syndrome] = step(crcDet, received);
57
58     disp('=====');
59     disp(['Синдром ошибки: ', num2str(syndrome)]);
60
61     n = length(encoded);
62     syndrome_table = zeros(n, 1);
63
64     disp('=====');
65     disp('Таблица синдромов');
66
67     syndrome_table = zeros(n, 4);
68
69     for i = 1:n
70         err = zeros(n,1);
71         err(i) = 1;
72         test_word = bitxor(encoded, logical(err));
73         temp = step(crcGen, test_word(1:end-4));
74         syndrome = temp(end-3:end);
75         syndrome_table(i,:) = syndrome';
76     end
77
78     bit_number = n - err_pos + 1;
79
80     T = table((1:n)', syndrome_table, ...
81         'VariableNames', {'BitNumber', 'Syndrome'});
82
83     disp(T);
84     disp(['Ошибочный бит: ', num2str(bit_number)]);
85
86     data7 = de2bi(46, 7, 'left-msb');
87
88     parity = data7(1);
89
90     for i = 2:length(data7)
91         parity = bitxor(parity, data7(i));
92     end
93
94     xor_encoded = [parity data7];
95
96     disp('=====');
97     disp('XOR-кодирование');

```

```

98     disp(num2str(xor_encoded));
99
100     err1 = de2bi(2^(5-1), 8, 'left-msb');
101     recv1 = bitxor(xor_encoded, err1);
102     parity_check1 = mod(sum(recv1), 2);
103
104     disp('=====');
105     disp('Ошибка в одном бите');
106     disp('Полученное сообщение:');
107     disp(num2str(recv1));
108
109     if parity_check1 == 0
110         disp('Ошибка НЕ обнаружена');
111     else
112         disp('Ошибка обнаружена');
113     end
114
115     err2_dec = 2^(5-1) + 2^(3-1);
116     err2 = de2bi(err2_dec, 8, 'left-msb');
117     recv2 = bitxor(xor_encoded, err2);
118     parity_check2 = mod(sum(recv2), 2);
119
120     disp('=====');
121     disp('Ошибка в двух битах');
122     disp('Полученное сообщение:');
123     disp(num2str(recv2));
124
125     if parity_check2 == 0
126         disp('Ошибка НЕ обнаружена');
127     else
128         disp('Ошибка обнаружена');
129     end

```